

Projekt zaliczeniowy — Porównanie algorytmów ML i wybór najlepszego modelu

Kurs Data Science DS02-05-26 · LearnIT

Termin oddania: 5 października 2026

- [1. Cel projektu](#)
- [2. Wymagany pipeline — etapy projektu](#)
 - [Etap 1: Definicja problemu biznesowego](#)
 - [Etap 2: Eksploracyjna analiza danych \(EDA\)](#)
 - [Etap 3: Czyszczenie i przygotowanie danych](#)
 - [Etap 4: Podział danych](#)
 - [Etap 5: Modelowanie — porównanie algorytmów](#)
 - [Etap 6: Ewaluacja modeli](#)
- [3. Dobór progu klasyfikacji \(threshold\) — tylko dla zadań klasyfikacyjnych](#)
- [4. Wybór najlepszego modelu](#)
- [5. Struktura i forma oddania projektu](#)
- [6. Częste błędy, których należy unikać](#)
- [7. Pytania i konsultacje](#)

1. Cel projektu

Celem projektu jest zbudowanie **pełnego pipeline'u danowego** dla wybranego zadania — **klasyfikacji lub regresji** (zależnie od Twojego zbioru danych) — oraz porównanie kilku różnych algorytmów uczenia maszynowego i **uzasadniony wybór najlepszego modelu**.

Projekt ma odwzorowywać rzeczywisty proces pracy Data Scientist: od surowych danych, przez eksplorację i przygotowanie, po modelowanie, strojenie hiperparametrów, ewaluację i rekomendację biznesową.

Zbiory danych zostały już zaakceptowane indywidualnie — pracujesz na swoim zgłoszonym temacie.

2. Wymagany pipeline — etapy projektu

Etap 1: Definicja problemu biznesowego

- Jasno opisz, **co przewidujesz** i **dla czego to ma znaczenie biznesowe**.
- Jeśli to klasyfikacja: zdefiniuj, co oznacza **fałszywy pozytywny (FP)** i **fałszywy negatywny (FN)**, oraz który błąd jest kosztowniejszy dla biznesu (to będzie istotne przy doborze progu decyzyjnego).
- Jeśli to regresja: zdefiniuj, jaki błąd predykcji jest akceptowalny i czy przeszacowanie/niedoszacowanie ma różny koszt biznesowy.

Etap 2: Eksploracyjna analiza danych (EDA)

Wymagane elementy: - Statystyki opisowe wszystkich zmiennych (rozkłady, braki danych, wartości odstające). - Rozkład zmiennej celu (dla klasyfikacji: balans klas; dla regresji: rozkład wartości, skośność). - Wizualizacje: rozkłady zmiennych numerycznych (histogramy/boxploty), zmienne katégoryczne (barploty), macierz korelacji. - Wstępna analiza zależności zmiennych objaśniających ze zmienną celu. -

Etap 3: Czyszczenie i przygotowanie danych

- Obsługa braków danych (imputacja — z uzasadnieniem metody).
- Obsługa wartości odstających (decyzja: usuwać / capować / zostawić — z uzasadnieniem).
- Kodowanie zmiennych kategoriycznych (One-Hot / Target/Ordinal Encoding — uzasadnij wybór).
- Skalowanie zmiennych numerycznych tam, gdzie jest to wymagane przez algorytm.
- Feature engineering: co najmniej **2 nowe cechy** stworzone na bazie istniejących, z uzasadnieniem hipotezy.

Etap 4: Podział danych

- Podział na **train / validation / test** (np. 60/20/20) lub train/test + walidacja krzyżowa (k-fold, zalecane k=5 lub 10).
- Dla klasyfikacji: **stratyfikacja** podziału względem zmiennej celu — obowiązkowa przy niezbalansowanych klasach.
- Cały pipeline preprocessingu (imputacja, skalowanie, encoding) musi być **dopasowany (fit) wyłącznie na zbiorze treningowym** i zastosowany (transform) na walidacyjnym/testowym — unikamy data leakage. Zalecane użycie `sklearn.pipeline.Pipeline` / `ColumnTransformer`.

Etap 5: Modelowanie — porównanie algorytmów

Wytrenuj i porównaj **minimum 4 różne algorytmy**.

Dla klasyfikacji, np.: 1. Regresja logistyczna (jako baseline) 2. Drzewo decyzyjne 3. Random Forest lub inny model bagging 4. Gradient Boosting (XGBoost / LightGBM / GradientBoostingClassifier) 5. (Opcjonalnie) SVM, KNN, sieć neuronowa (MLP)

Dla regresji, np.: 1. Regresja liniowa (jako baseline) 2. Drzewo decyzyjne (regresor) 3. Random Forest Regressor 4. Gradient Boosting (XGBoost / LightGBM / GradientBoostingRegressor) 5. (Opcjonalnie) SVR, KNN Regressor, sieć neuronowa (MLP)

Dla każdego modelu: - Krótki opis działania algorytmu (2-3 zdania) — pokazuje, że rozumiesz nie tylko `model.fit()`. - **Strojenie hiperparametrów** (GridSearchCV / RandomizedSearchCV / Optuna) na zbiorze treningowym z walidacją krzyżową. - Zapisz najlepsze hiperparametry i uzasadnij zakres przeszukiwania.

Etap 6: Ewaluacja modeli

Dla każdego z modeli zestaw w jednej tabeli porównawczej odpowiednie metryki:

Klasyfikacja (na progu domyślnym 0.5 — punkt wyjścia): - Accuracy, Precision, Recall, F1-score - ROC-AUC oraz PR-AUC (**PR-AUC jest ważniejszy przy niezbalansowanych klasach** — wyjaśnij dlaczego) - Macierz pomyłek (confusion matrix) - Krzywe ROC i Precision-Recall dla wszystkich modeli na jednym wykresie

Regresja: - MAE, RMSE, MAPE, R^2 - Wykres rzeczywiste vs. przewidywane wartości - Analiza reszt (residual plot) — sprawdź, czy błędy są losowe czy mają wzorec (heteroskedastyczność, nieliniowość)

3. Dobór progu klasyfikacji (threshold) — tylko dla zadań klasyfikacyjnych

Jeśli Twój problem to klasyfikacja, jest to kluczowy, obowiązkowy element projektu. Próg 0.5 jest tylko punktem wyjścia — musisz świadomie dobrać próg optymalny dla problemu biznesowego zdefiniowanego w Etapie 1.

Krok 1 — Wygeneruj predykcje prawdopodobieństw Dla najlepszego kandydata (lub top 2–3 modeli) użyj `predict_proba()` (nie `predict()`) na zbiorze walidacyjnym, aby uzyskać ciągłe prawdopodobieństwa, a nie twarde etykiety.

Krok 2 — Zbuduj krzywą Precision-Recall vs. threshold Narysuj wykres, na którym oś X to threshold (0 do 1), a oś Y to jednocześnie Precision, Recall i F1-score. Pozwala to wizualnie zobaczyć punkt kompromisu (trade-off).

Krok 3 — Zdefiniuj kryterium optymalizacji progów, jedno z poniższych (dobierz zgodnie z Etapem 1): - **Maksymalizacja F1-score** — gdy FP i FN są równie ważne. - **Maksymalizacja F-beta score** (beta > 1 gdy Recall ważniejszy, beta < 1 gdy Precision ważniejszy) — użyj, gdy koszty FP/FN są asymetryczne. - **Youden's J statistic** (na krzywej ROC: $J = TPR - FPR$) — dobre uniwersalne podejście, gdy nie masz jawnej funkcji kosztu. - **Podejście kosztowe (cost-sensitive)**: przypisz koszt pieniężny/biznesowy do FP i FN i wybierz próg minimalizujący **oczekiwany koszt całkowity** = $FP \times \text{koszt_FP} + FN \times \text{koszt_FN}$. **To podejście jest zalecane**, jeśli tylko potrafisz sensownie oszacować koszty. - **Ograniczenie na jedną z metryk** — np. "maksymalizuj Recall przy Precision $\geq 80\%$ ".

Krok 4 — Wyznacz optymalny threshold na zbiorze **walidacyjnym** (nigdy na testowym!) zgodnie z wybranym kryterium z Kroku 3. Pokaż to w kodzie (np. iteracja po siatce thresholdów co 0.01 i wybór argmax/argmin funkcji celu).

Krok 5 — Zweryfikuj wybrany próg na zbiorze testowym Zastosuj wyznaczony próg (nie 0.5!) do predykcji na zbiorze testowym i policz finalne metryki. Porównaj wynik z metrykami przy progu 0.5 — pokaż, o ile poprawiła się metryka istotna biznesowo.

Krok 6 — Uzasadnienie biznesowe Napisz 3–5 zdań: dlaczego wybrany próg (a nie 0.5) jest lepszy z perspektywy biznesowej zdefiniowanej w Etapie 1.

4. Wybór najlepszego modelu

- Zestaw finalną tabelę porównawczą wszystkich modeli (dla klasyfikacji: po doborze ich własnych progów, nie tylko na 0.5).
- Wskaż **jeden model finalny** i uzasadnij wybór biorąc pod uwagę:
 - metryki na zbiorze testowym,
 - stabilność wyników (np. wariancja w walidacji krzyżowej),
 - interpretowalność (czy w Twoim przypadku biznesowym jest istotna),
 - koszt obliczeniowy / czas predykcji (istotne przy wdrożeniu),
 - ważność cech (feature importance / SHAP) dla wybranego modelu — pokaż top 10 cech i skomentuj, czy mają sens biznesowy.

5. Struktura i forma oddania projektu

Projekt oddajesz jako: 1. **Jupyter Notebook** (`.ipynb`) z pełnym, uruchamialnym kodem, jasno podzielonym na sekcje: **EDA** → **przygotowanie danych** → **modelowanie** → **dobór hiperparametrów** → **ewaluacja/dobór progów** → **wybór finalnego modelu** (nagłówki Markdown). 2. **Krótkie podsumowanie (1-2 strony, PDF lub Markdown)** — executive summary: problem, wybrany model, kluczowe metryki (i próg — jeśli klasyfikacja), rekomendacja biznesowa. Ma być zrozumiałe dla osoby nietechnicznej. 3. Notebook musi się w pełni wykonać od góry do dołu bez błędów (`Kernel` → `Restart & Run All`) przed oddaniem.

Termin oddania: niedziela, 5 października 2026, do godziny 23:59. Pliki wysyłasz na maila/platformę kursu zgodnie z dotychczasowymi ustaleniami. Spóźnienia bez wcześniejszego zgłoszenia — zgodnie z zasadami kursu.

6. Częste błędy, których należy unikać

- Fitowanie scalera/imputera/encodera na całym zbiorze danych przed podziałem train/test (data leakage).
 - Klasyfikacja: ocena modelu tylko za pomocą Accuracy przy niezbalansowanych klasach.
 - Klasyfikacja: dobór progu na zbiorze testowym zamiast walidacyjnym (przeuczenie progu do danych testowych).
 - Klasyfikacja: porównywanie modeli wyłącznie na domyślnym progu 0.5, bez uwzględnienia że różne modele mają różne optymalne progi.
 - Regresja: poleganie wyłącznie na R^2 bez sprawdzenia reszt (residuals) i błędów bezwzględnych (MAE/RMSE).
 - Brak uzasadnienia biznesowego podjętych decyzji modelowych.
-

7. Pytania i konsultacje

W razie pytań dotyczących zakresu projektu lub konkretnych decyzji modelowych — pisz bezpośrednio do prowadzącego.

Powodzenia!